

Serverless API (AWS Lambda) – Documentation

1. Project Title

Serverless Employee Management API Using AWS Lambda and Amazon API Gateway

2. Objective

The objective of this project is to develop a serverless REST API using AWS Lambda and Amazon API Gateway. The project demonstrates how to create and deploy a serverless application that provides employee-related information through multiple API endpoints without managing any servers.

3. AWS Services Used

- AWS Lambda
- Amazon API Gateway
- Amazon CloudWatch

4. Project Details

Lambda Function Name: EmployeeAPI

API Type: HTTP API

Stage: \$default

Runtime: Python 3.x

Authentication: None

API Endpoints:

- GET /employees
- GET /employees/{id}
- GET /departments
- GET /company
- GET /health

5. Project Architecture

Client (Browser)

↓

Amazon API Gateway (HTTP API)

↓

AWS Lambda (EmployeeAPI)

↓

JSON Response

6. Steps Followed to Develop the API

Step 1: Create the Lambda Function

- Opened the AWS Lambda Console.
- Clicked **Create Function**.
- Selected **Author from Scratch**.
- Entered **EmployeeAPI** as the function name.
- Selected **Python 3.x** as the runtime.
- Created the Lambda function with the default execution role.

Step 2: Develop the Lambda Function

- Replaced the default Lambda code with a Python script.
- Created sample employee data in JSON format.
- Added routing logic to process multiple API requests.
- Configured the function to return JSON responses for different endpoints.
- Deployed the updated Lambda function.

Step 3: Create an HTTP API

- Opened the Amazon API Gateway Console.
- Created a new HTTP API.
- Selected the EmployeeAPI Lambda function as the integration.
- Enabled automatic deployment using the **\$default** stage.

Step 4: Configure API Routes

Created the following routes:

- GET /employees
- GET /employees/{id}
- GET /departments
- GET /company
- GET /health

Associated all routes with the EmployeeAPI Lambda function.

Step 5: Test the API

Obtained the API Invoke URL.

Tested the following endpoints using a web browser:

- GET /employees
- GET /employees/1
- GET /departments
- GET /company
- GET /health

Verified that each endpoint returned the expected JSON response.

7. API Responses

Get All Employees

Returns a list of all employee records in JSON format.

Get Employee by ID

Returns the details of a specific employee based on the employee ID.

Get Departments

Returns the list of departments available in the organization.

Get Company Details

Returns general company information.

Health Check

Returns the API status and version information.

8. Output

- Successfully created a serverless REST API using AWS Lambda.
- Configured Amazon API Gateway to expose multiple API endpoints.
- Implemented routing for different API requests within a single Lambda function.
- Returned structured JSON responses for all endpoints.
- Verified successful communication between API Gateway and AWS Lambda.
- Confirmed API functionality through browser-based testing.

9. Conclusion

This project demonstrates the implementation of a serverless Employee Management REST API using AWS Lambda and Amazon API Gateway. A single Lambda function was integrated with multiple API routes to provide employee, department, company, and health information in JSON format. The project showcases the use of AWS serverless services to build scalable, cost-effective, and event-driven applications without managing server infrastructure.